

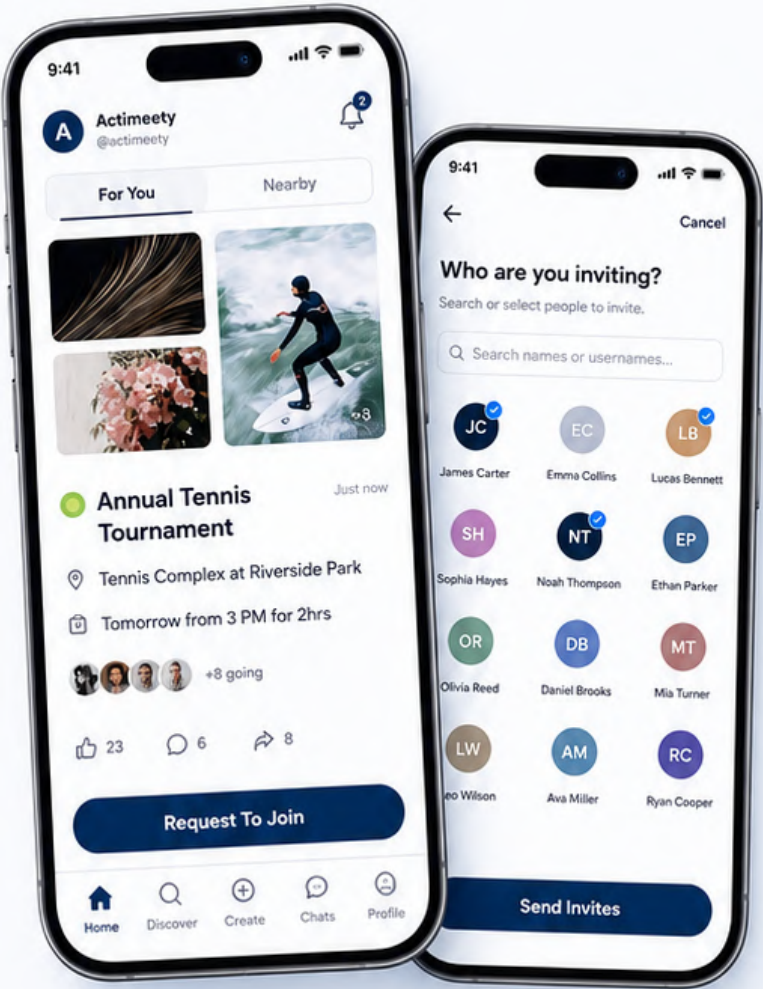
Frontend Case Study

Product flow, architecture, rendering strategy, performance, and UI execution.

Actimeety

Social Activity Platform

Actimeety helps users discover activities, create plans, invite others, request to join, and coordinate through real-time social interactions.



Overview

- Activity-based social networking with feed, discovery, profiles, comments, messaging, maps, and activity creation.
- Frontend focused on real-time feed discovery, partner finding, and mobile-first interaction quality.
- Architecture optimized for scalable feature boundaries, performance, and future evolution inside an Nx monorepo.

Feed

Discovery

Creation

Invites

Chat

Maps



My Role

Senior Frontend Engineer /
Lead Frontend Engineer.

Owned frontend architecture,
product UI delivery, performance,
and cross-team collaboration.



Frontend Stack

Next.js, React, TypeScript,
Nx Monorepo, Apollo GraphQL,
GraphQL Subscriptions,
React Hook Form, Zustand,
MUI, Storybook, PWA, Leaflet.



Backend Context

NestJS microservices via
GraphQL Federation, backed by
MongoDB, PostgreSQL + PostGIS,
Neo4j, Kafka, BullMQ, and
Kubernetes.



Frontend Proof Points



Real-time surfaces

Feed freshness, subscriptions,
chat interactions, and live
coordination.



Component system

Customized MUI, Storybook,
compound components, and
reusable hooks.



Mobile-first PWA

Responsive layouts and app-like
interaction patterns for mobile
delivery.

Vision & User Flow

The frontend translated an activity-based social idea into a guided, real-time, mobile-first experience.

What Actimeety Solves

Most social platforms connect people through content alone. Actimeety connects people through intent, shared interests, free time, nearby opportunities, and real coordination before the activity happens.

Primary User Journey

- 1

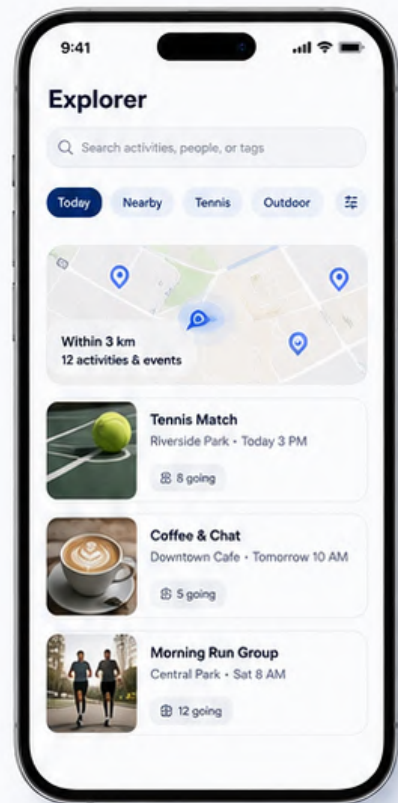
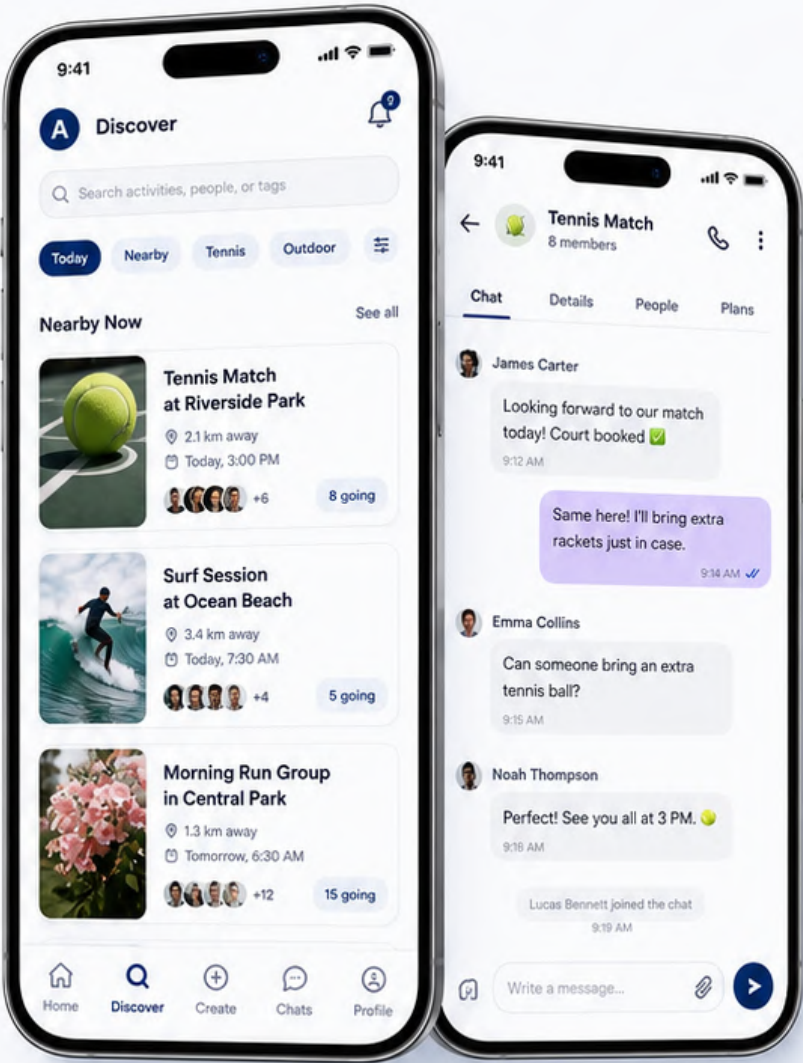
Discover activities in the real-time feed
- 2

Search activities, people, or tags in Discovery
- 3

Filter by time, location, category, or tags
- 4

Create an activity with a guided multi-step flow
- 5

Request to join, invite others, and coordinate in chat



Explorer & Discovery

Discovery is driven by search, category filters, location-aware results, tags, and relevance-based activity lists.

- Time
- Location
- Category
- Tags
- People
- Activities

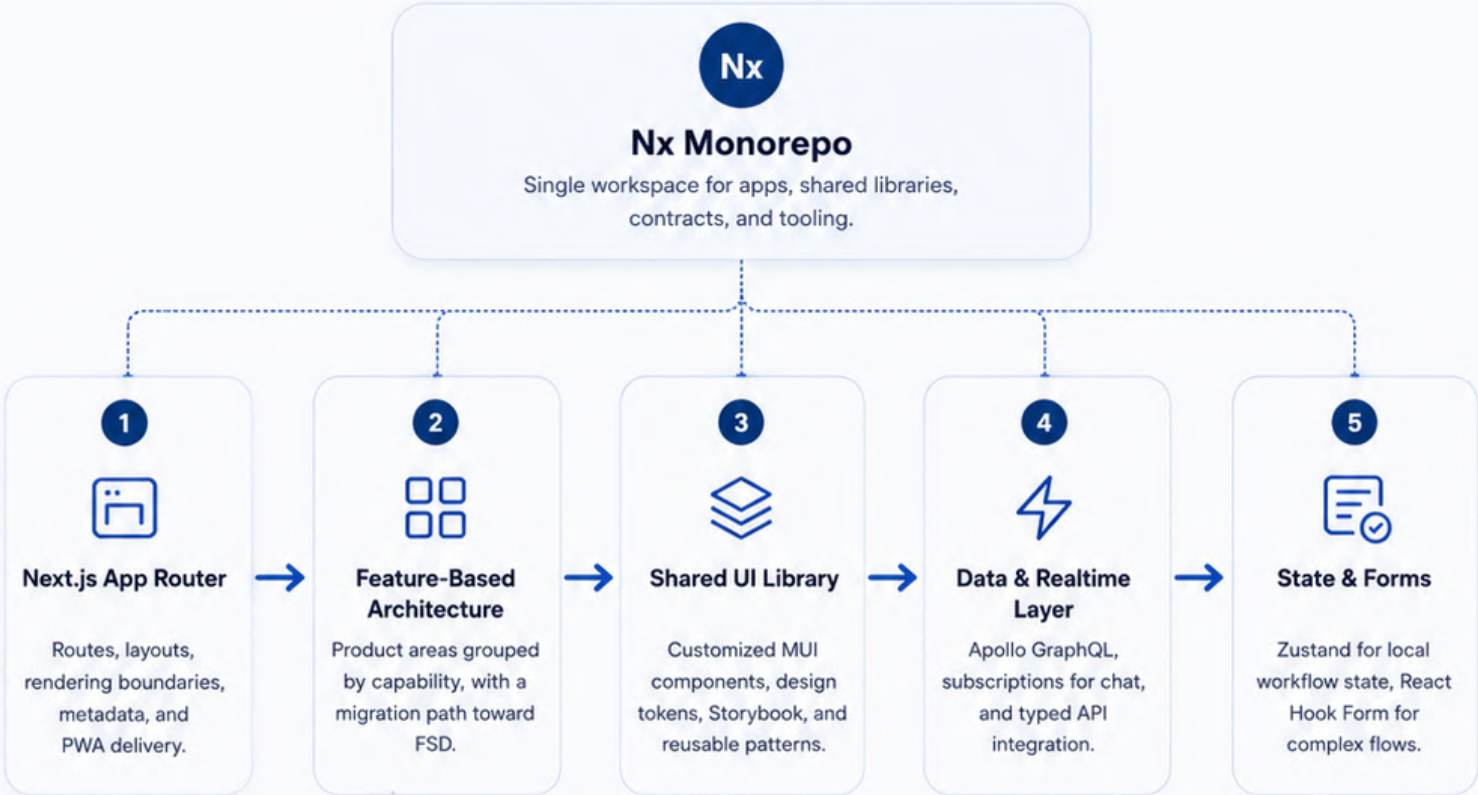


How the Feed Works

- Ranks activities using relevance signals.**
 Interests, activity preferences, following relationships, free time, and nearby location shape what you see first.
- Real-time freshness with seamless pagination.**
 Cursor-based pagination and real-time updates ensure new relevant activities appear without breaking your scroll.
- Balances discovery, social proof, and action.**
 The feed surfaces the right mix of nearby activities, people going, and intent—so you can act, not just browse.

Frontend Architecture

A feature-oriented Next.js architecture inside an Nx monorepo, designed for maintainability, performance, and future evolution.



Architecture Goals

- 1

Maintainable boundaries
Keep UI, business logic, data access, and shared components separated.
- 2

Future-ready structure
Feature-oriented organization with an evolution path toward FSD and microfrontends if needed.
- 3

Rendering per surface
Use different rendering strategies for feed, discovery, chat, profiles, and forms.
- 4

Developer experience
Storybook, shared patterns, and reusable hooks for faster, more consistent delivery.

Backend Boundary

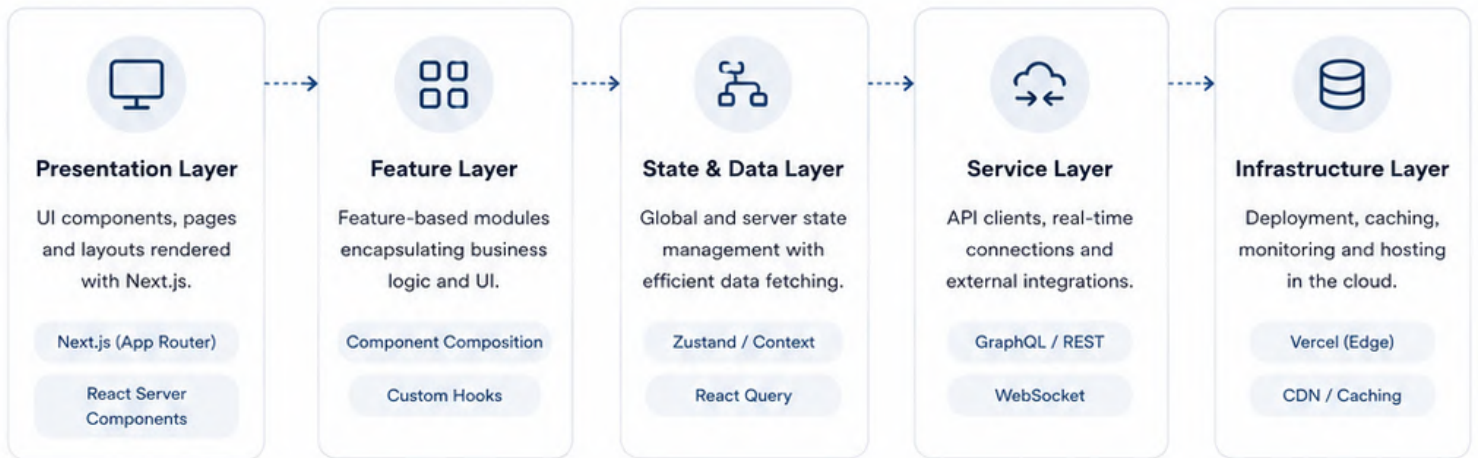
The frontend consumed a unified GraphQL layer backed by NestJS microservices, while backend services owned domains such as auth, users, activities, feed, social graph, places, messenger, notifications, and search.

Architecture & Performance

Actimeety is built with a scalable, modular architecture and modern frontend techniques to deliver a fast, reliable and real-time social experience.



Frontend Architecture



Backend (At a Glance)

The backend is powered by a scalable microservices architecture built with NestJS, GraphQL Federation, PostgreSQL, MongoDB, Redis, Kafka and deployed on Kubernetes.

[Learn more in backend case study →](#)



Performance Highlights



92+

Lighthouse Performance Score

Achieved through code splitting, caching and rendering optimizations.



+68%

Faster Perceived Load Time

Hybrid rendering, streaming and prefetching reduce wait times.



-45%

JavaScript Bundle Size

Route-level code splitting and tree shaking cut unused code.



100%

Core Web Vitals Passed

Optimized LCP, FID and CLS for a smooth user experience.



Key Technical Achievements



Scalable Component System

Built a reusable, accessible and themeable component library that accelerates feature delivery.



Real-time Experience

Integrated WebSockets for live interactions, notifications and activity updates.



Advanced Data Strategies

Implemented multi-layer caching, stale-while-revalidate and intelligent data prefetching.



Developer Experience

Standardized project structure, linting, testing and Storybook to improve collaboration and consistency.



Reliability & Observability

Error boundaries, performance monitoring and analytics ensure a stable and observable application.